

6. PROBLEMAS DE SATISFACCIÓN DE RESTRICCIONES

(PARTE 1)

IA 3.2 - Programación III

1º C - 2025

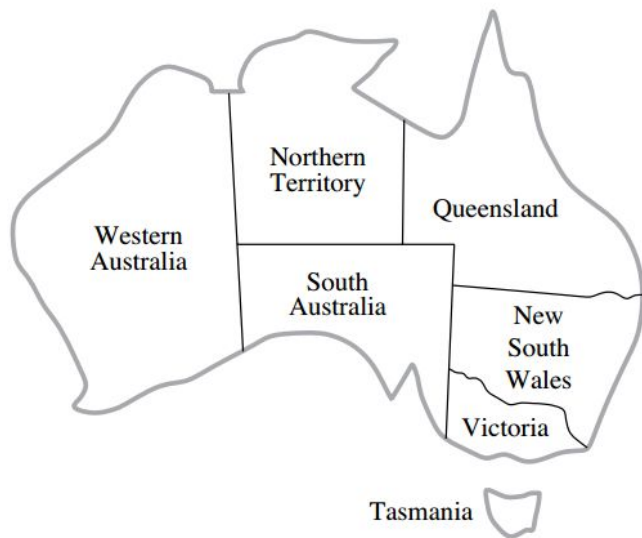
Dr. Mauro Lucci

Motivación

Problema de coloreo del mapa de Australia

— — —

Objetivo. Colorear cada región de Australia con rojo, verde o azul de manera que ningún par de regiones limítrofes tenga un mismo color.





Desafío

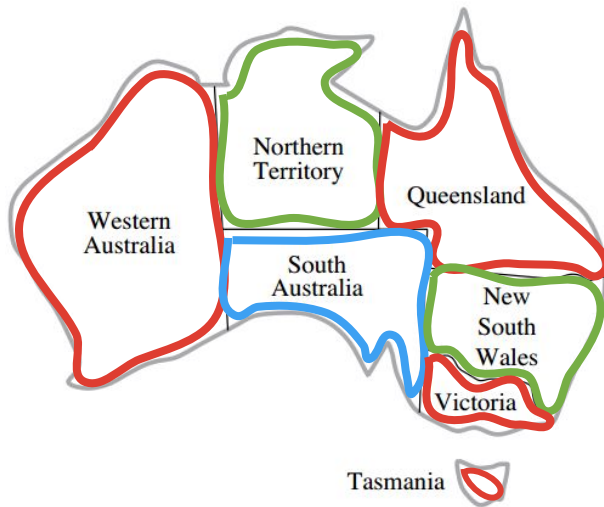
— — —

¿Es posible resolver este problema?



Respuesta

Existen muchas soluciones para este problema. Una de ellas:

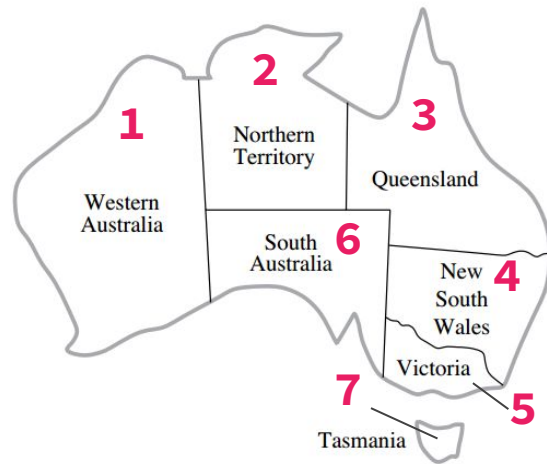


Formulación incremental

A partir de lo visto en la materia, para resolverlo debemos formularlo como un **problema de búsqueda** y aplicar alguno de los **algoritmos de búsqueda** vistos.

Veamos por ejemplo una posible formulación incremental.

Asumiremos que las regiones tienen un **orden**. En nuestro caso será:



Formulación incremental

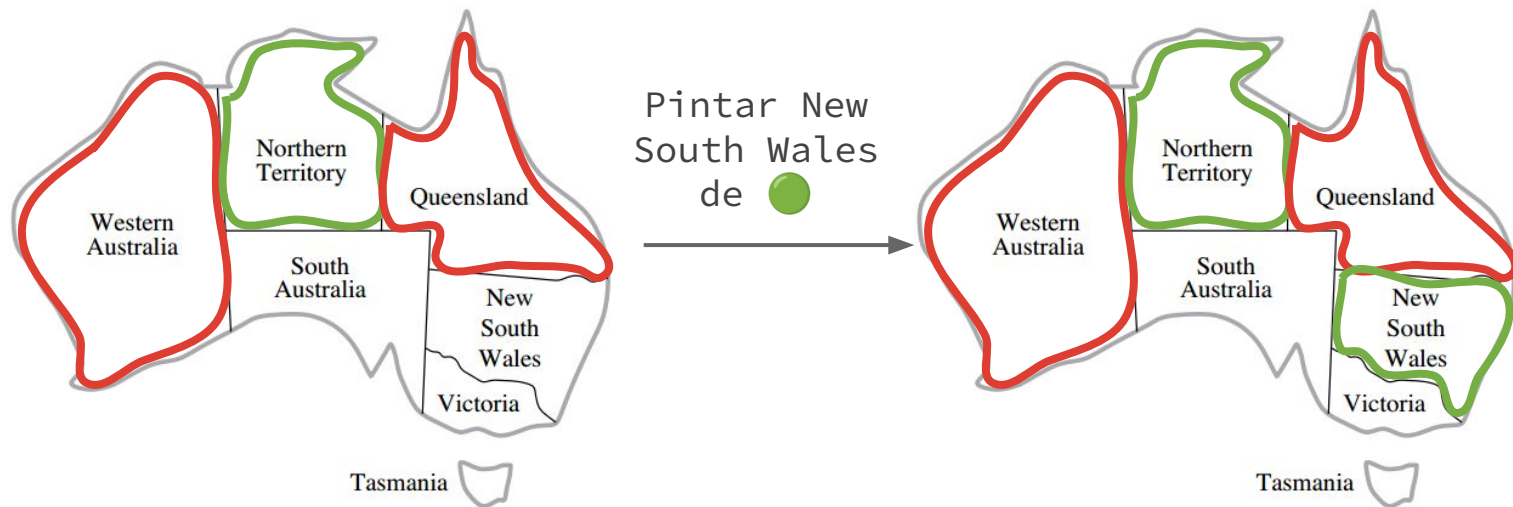
Un **estado** es un tupla vacía $()$ o una tupla $s = (s_1, \dots, s_j)$, con $j \in \{1, \dots, 7\}$, donde para cada $i \in \{1, \dots, j\}$, $s_i \in \{\text{red}, \text{green}, \text{blue}\}$ es el color de la i -ésima región.

El número total de estados es $1 + 3 + 3^2 + \dots + 3^7 = \mathbf{3280}$.

Una **acción** es un color $x \in \{\text{red}, \text{green}, \text{blue}\}$ que representa pintar la primer región no coloreada con x .



Ejemplo



Representación.



Formulación incremental

— — —

- **Estado inicial.** La tupla vacía $()$.

- **Acciones.** Dado un estado s ,

$$\text{ACCIONES}(s) = \begin{cases} \{\text{red}, \text{green}, \text{blue}\} & \text{si } s = () \text{ o } s = (s_1, \dots, s_j) \text{ con } j < 7. \\ \{\} & \text{si } s = (s_1, \dots, s_7). \end{cases}$$

- **Modelo de transiciones.** Dado un estado s y una acción x ,

$$\text{RESULTADO}(s, x) = \begin{cases} (x) & \text{si } s = () \\ (s_1, \dots, s_j, x) & \text{si } s = (s_1, \dots, s_j) \text{ con } j < 7 \end{cases}$$

- **Test objetivo.** Un estado $(s_1, \dots, s_7)$ es objetivo si todo par de regiones limítrofes tiene colores distintos.
- **Costo de camino.** Es la suma de los costos individuales de las acciones que realiza. El costo individual de cada acción es 1.

Formulación incremental

— — —

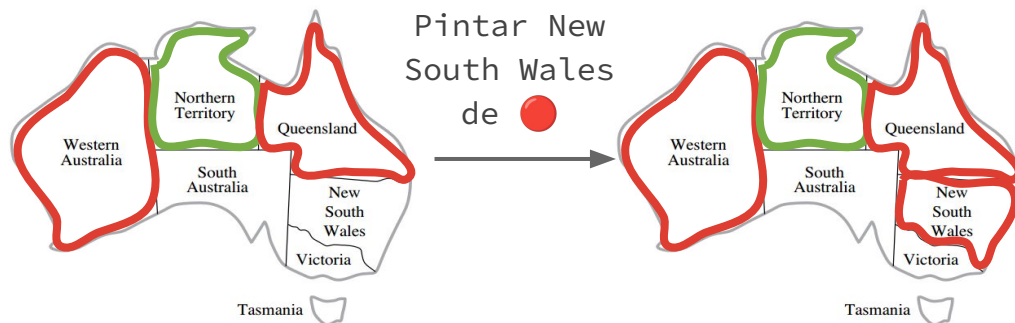
Pregunta. ¿Qué inconveniente tiene esta formulación?

Formulación incremental

— — —

Pregunta. ¿Qué inconveniente tiene esta formulación?

Respuesta. Permite pintar una región del mismo color que una región limítrofe.

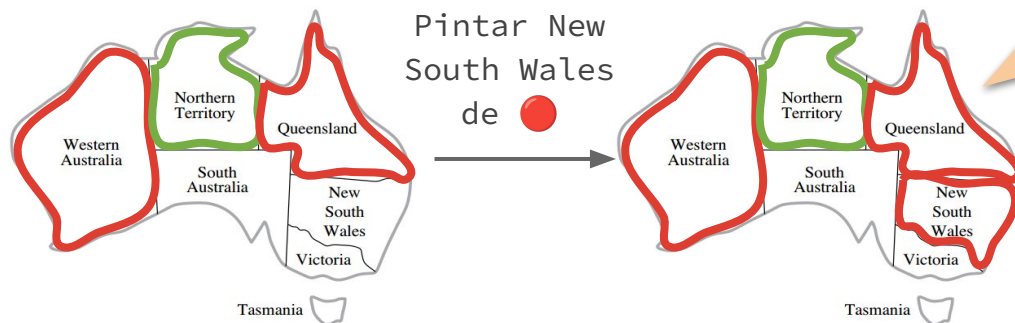


Representación. (●, ●, ●) → (●, ●, ●, ●)

Formulación incremental

Pregunta. ¿Qué inconveniente tiene esta formulación?

Respuesta. Permite pintar una región del mismo color que una región limítrofe.



No tiene sentido seguir expandiendo un nodo que contenga tal estado.

¿Cómo lo arreglamos? 🤔

Representación.

(●, ●, ●)



(●, ●, ●, ●)

Formulación incremental mejorada

Nos interesan los estados que cumplan la siguiente **restricción**:

Para todo par de regiones limítrofes, si están pintadas, tienen colores distintos.

Para cada $i \in \{1, \dots, 7\}$, sea $N(i)$ el conjunto de índices de las regiones limítrofes con la región i . Dado un estado $(s_1, \dots, s_j)$ podemos expresar matemáticamente esta restricción como:

Para todo $i, k \in \{1, \dots, j\}$ tal que $i < k$ y $k \in N(i)$, $s_i \neq s_k$.

Formulación incremental mejorada

Reformulamos el problema para que aquellos estados que violen esta restricción sean **inalcanzables**.

ACCIONES(s) =

$$\begin{cases} \{\text{red}, \text{green}, \text{blue}\} & \text{si } s = () \\ \{\text{red}, \text{green}, \text{blue}\} \setminus \underbrace{\{s_i : i \in N(j+1)\}}_{\substack{\text{Conjunto de colores} \\ \text{usados por las regiones} \\ \text{límitrofes a } j+1}} & \text{si } s = (s_1, \dots, s_j) \text{ con } j < 7. \\ \{\} & \text{si } s = (s_1, \dots, s_7). \end{cases}$$



Próximamente

— — —

A continuación, veremos un nuevo formalismo para modelar **problemas de factibilidad** mediante **variables** con toman valores en ciertos **dominios** y donde las soluciones son asignaciones de valores a variables sujetas a un conjunto de **restricciones**.

Esto permitirá definir **nuevos algoritmos de búsqueda que incorporan heurísticas generales** (que no dependen del problema particular a resolver).

Problema de Satisfacción de Restricciones

Constraint Satisfaction Problem (CSP)

Un **Problema de Satisfacción de Restricciones (CSP)** se compone de:

1. Un conjunto de **variables**

$$X = \{X_1, \dots, X_n\}.$$

2. Un conjunto de **dominios**

$$D = \{D_1, \dots, D_n\}.$$

Cada dominio $D_i = \{v_{i1}, \dots, v_{ik}\}$ es el conjunto de valores permitidos para la variable X_i .

3. Un conjunto de **restricciones** que especifica las combinaciones de valores permitidas.

$$C = \{C_1, \dots, C_r\}.$$

— — —

Restricciones

Cada restricción C_i de C es un par $\langle Y_i, R_i \rangle$ donde:

- Y_i es una tupla de **variables participantes** en la restricción,
- R_i es una **relación** con los valores que esas variables pueden tomar.

 **Ejemplo.** Si las variables X_1 y X_2 tienen dominio $D_1 = D_2 = \{0,1\}$, entonces una restricción que diga que X_1 y X_2 deben tener valores diferentes se escribe:

$\langle (X_1, X_2), X_1 \neq X_2 \rangle$ o bien $\langle (X_1, X_2), \{(0,1), (1,0)\} \rangle$

Relación dada **simbólicamente**.

Relación dada por **extensión**.



Ejemplo – Coloreo de Australia

Variables.

$$X = \{W, N, Q, NS, V, S, T\}.$$

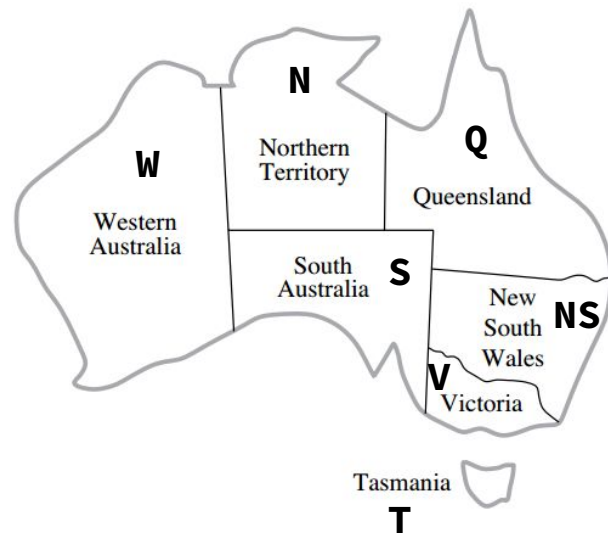
Dominios.

$$D_i = \{\text{red}, \text{green}, \text{blue}\} \text{ para cada variable } i \in X.$$

Restricciones.

$$C = \{S \neq W, S \neq N, S \neq Q, S \neq NS, S \neq V, W \neq N, N \neq Q, Q \neq NS, NS \neq V\}$$

La restricción $S \neq W$ es una **abreviatura** para
 $\langle (S, W), S \neq W \rangle$ o equivalentemente
 $\langle (S, W), \{(\text{red}, \text{green}), (\text{red}, \text{blue}), (\text{green}, \text{red}), (\text{green}, \text{blue}), (\text{blue}, \text{red}), (\text{blue}, \text{green})\} \rangle$





Ejemplo – Scheduling

— — —



Una parte del proceso de ensamblaje de un auto requiere de la siguiente secuencia de tareas:

1. Instalar tren delantero y trasero (duración: 10 minutos por tren).
2. Colocar ruedas (duración: 1 minuto por rueda).
3. Ajustar las tuercas de cada rueda (duración: 2 minutos por rueda).
4. Colocar la taza de cada rueda (duración: 1 minuto por rueda).
5. Inspeccionar el ensamblaje final (duración: 3 minutos).

Objetivo. Programar la hora de inicio de cada tarea, respetando el orden de las mismas, en un tiempo límite de 30 min.



Ejemplo – Scheduling

— — —

Variables. Definimos una variable por cada tarea:

TREN $\uparrow$: Hora de inicio de la instalación del tren delantero.

TREN $\downarrow$: Hora de inicio de la instalación del tren trasero.

RUEDA κ : Hora de inicio de la colocación de la rueda delantera izquierda.

... y así sucesivamente tenemos:

$X = \{\text{TREN}\uparrow, \text{TREN}\downarrow, \text{RUEDA}\kappa, \text{RUEDA}\nearrow, \text{RUEDA}\searrow, \text{RUEDA}\swarrow, \text{TUERCA}\kappa, \text{TUERCA}\nearrow, \text{TUERCA}\searrow, \text{TUERCA}\swarrow, \text{TAZA}\kappa, \text{TAZA}\nearrow, \text{TAZA}\searrow, \text{TAZA}\swarrow, \text{INSPEC}\}.$



Ejemplo – Scheduling

— — —

Dominios. Suponiendo que el ensamblaje debe completarse en un tiempo máximo de 30 minutos, podemos pensar que el dominio de cada variable $i \in X$ es:

$$D_i = \{0, 1, \dots, 30\}.$$

Por ejemplo, $TREN_{\uparrow} = 5$ indica que el tren delantero se comienza a instalar en el minuto 5.



Ejemplo – Scheduling

— — —

Restricciones. Necesitamos establecer precedencia entre las variables.

Por ejemplo, necesitamos expresar que las ruedas delanteras se pueden comenzar a colocar luego de que el tren delantero se haya terminado de instalar. ¿Cómo hacemos? 🤔



Ejemplo – Scheduling

— — —

Restricciones. Necesitamos establecer precedencia entre las variables.

Por ejemplo, necesitamos expresar que las ruedas delanteras se pueden comenzar a colocar luego de que el tren delantero se haya terminado de instalar. ¿Cómo hacemos? 🤔

$$\text{TREN}\uparrow + 10 \leq \text{RUEDA}\kappa$$

$$\text{TREN}\uparrow + 10 \leq \text{RUEDA}\nearrow$$

$\text{TREN}\uparrow + 10$ es la hora en que termina de instalarse el tren delantero.

Este tipo de restricciones se conocen como **restricciones de precedencia**.



Ejemplo – Scheduling

— — —

Recordar que:

$$\text{TREN}\uparrow + 10 \leq \text{RUEDA}\kappa$$

es una abreviatura de la restricción:

$$\langle (\text{TREN}\uparrow, \text{RUEDA}\kappa), \text{TREN}\uparrow + 10 \leq \text{RUEDA}\kappa \rangle$$

o equivalentemente:

$$\langle (\text{TREN}\uparrow, \text{RUEDA}\kappa), \{ (0, 10), (0, 11), (0, 12), \dots, (0, 30), (1, 11), (1, 12), \dots, (1, 30), \dots \} \rangle.$$



Ejemplo – Scheduling

Tenemos entonces:

$$\text{TREN}\uparrow + 10 \leq \text{RUEDA}\kappa$$

$$\text{TREN}\downarrow + 10 \leq \text{RUEDA}\swarrow$$

$$\text{RUEDA}\kappa + 1 \leq \text{TUERCA}\kappa$$

$$\text{RUEDA}\swarrow + 1 \leq \text{TUERCA}\swarrow$$

$$\text{TUERCA}\kappa + 2 \leq \text{TAZA}\kappa$$

$$\text{TUERCA}\swarrow + 2 \leq \text{TAZA}\swarrow$$

$$\text{TAZA}\kappa + 1 \leq \text{INSPEC}$$

$$\text{TAZA}\swarrow + 1 \leq \text{INSPEC}$$

$$\text{TREN}\uparrow + 10 \leq \text{RUEDA}\nearrow$$

$$\text{TREN}\downarrow + 10 \leq \text{RUEDA}\searrow$$

$$\text{RUEDA}\nearrow + 1 \leq \text{TUERCA}\nearrow$$

$$\text{RUEDA}\searrow + 1 \leq \text{TUERCA}\searrow$$

$$\text{TUERCA}\nearrow + 2 \leq \text{TAZA}\nearrow$$

$$\text{TUERCA}\searrow + 2 \leq \text{TAZA}\searrow$$

$$\text{TAZA}\nearrow + 1 \leq \text{INSPEC}$$

$$\text{TAZA}\searrow + 1 \leq \text{INSPEC}$$

¿Falta alguna? 🤔



Ejemplo – Scheduling

— — —

Nos falta decir que el ensamblaje finaliza en a lo sumo 30 minutos. ¿Cómo lo hacemos? 🤔



Ejemplo – Scheduling

Nos falta decir que el ensamblaje finaliza en a lo sumo 30 minutos. ¿Cómo lo hacemos? 🤔

$$\text{INSPEC} + 3 \leq 30$$

Es decir,

$$\text{INSPEC} \leq 27$$

Notar que involucra una única variable y que es una abreviatura de la restricción:

$$\langle (\text{INSPEC}), \text{INSPEC} \leq 27 \rangle$$



Ejemplo – Scheduling

— — —

Ahora... Una de las máquinas instaladora de trenes queda fuera de servicio y los operadores ya no pueden instalar los trenes delantero y trasero simultáneamente, sino que deben hacerlo uno después del otro. ¿Cómo incorporamos esta restricción? 🤔



Ejemplo – Scheduling

— — —

Ahora... Una de las máquinas instaladora de trenes queda fuera de servicio y los operadores ya no pueden instalar los trenes delantero y trasero simultáneamente, sino que deben hacerlo uno después del otro. ¿Cómo incorporamos esta restricción? 🤔

$$(\text{TREN}\uparrow + 10 \leq \text{TREN}\downarrow) \textbf{ or } (\text{TREN}\downarrow + 10 \leq \text{TREN}\uparrow)$$

Este tipo de restricciones se conoce como **restricciones disyuntivas** (en este caso particular, entre dos restricciones de precedencia).



Ejemplo – Scheduling

Recordar que:

$$(\text{TREN}\uparrow + 10 \leq \text{TREN}\downarrow) \textbf{ or } (\text{TREN}\downarrow + 10 \leq \text{TREN}\uparrow)$$

es una abreviatura de la restricción:

$$\langle (\text{TREN}\uparrow, \text{TREN}\downarrow), (\text{TREN}\uparrow + 10 \leq \text{TREN}\downarrow) \textbf{ or } (\text{TREN}\downarrow + 10 \leq \text{TREN}\uparrow) \rangle$$

o equivalentemente:

$$\begin{aligned} & \langle (\text{TREN}\uparrow, \text{TREN}\downarrow), \\ & \{ (0, 10), (10, 0), (0, 11), (11, 0), \dots, (1, 11), (11, 1), (1, 12), (12, 1), \dots \} \rangle. \end{aligned}$$

Esta restricción puede parecer más compleja... pero no es más que una relación binaria que se puede dar por extensión.



Ejemplo – Scheduling

— — —

- **Variables.**

$X = \{\text{TREN}\uparrow, \text{TREN}\downarrow, \text{RUEDA}\kappa, \text{RUEDA}\nearrow, \text{RUEDA}\searrow, \text{RUEDA}\swarrow, \text{TUERCA}\kappa, \text{TUERCA}\nearrow, \text{TUERCA}\searrow, \text{TUERCA}\swarrow, \text{TAZA}\kappa, \text{TAZA}\nearrow, \text{TAZA}\searrow, \text{TAZA}\swarrow, \text{INSPEC}\}$

- **Dominios.**

$D_i = \{0, 1, \dots, 30\}$ para cada variable $i \in X$.

- **Restricciones.**

$$\text{TREN}\uparrow + 10 \leq \text{RUEDA}\kappa$$

$$\text{TREN}\downarrow + 10 \leq \text{RUEDA}\searrow$$

$$\text{RUEDA}\swarrow + 1 \leq \text{TUERCA}\swarrow$$

$$\text{TUERCA}\nearrow + 2 \leq \text{TAZA}\nearrow$$

$$\text{TAZA}\kappa + 1 \leq \text{INSPEC}$$

$$\text{TAZA}\searrow + 1 \leq \text{INSPEC}$$

$$\text{TREN}\uparrow + 10 \leq \text{RUEDA}\nearrow$$

$$\text{RUEDA}\kappa + 1 \leq \text{TUERCA}\kappa$$

$$\text{RUEDA}\searrow + 1 \leq \text{TUERCA}\searrow$$

$$\text{TUERCA}\swarrow + 2 \leq \text{TAZA}\swarrow$$

$$\text{TAZA}\nearrow + 1 \leq \text{INSPEC}$$

$$\text{INSPEC} \leq 27$$

$$\text{TREN}\downarrow + 10 \leq \text{RUEDA}\swarrow$$

$$\text{RUEDA}\nearrow + 1 \leq \text{TUERCA}\nearrow$$

$$\text{TUERCA}\kappa + 2 \leq \text{TAZA}\kappa$$

$$\text{TUERCA}\searrow + 2 \leq \text{TAZA}\searrow$$

$$\text{TAZA}\swarrow + 1 \leq \text{INSPEC}$$

$$(\text{TREN}\uparrow + 10 \leq \text{TREN}\downarrow) \text{ or } (\text{TREN}\downarrow + 10 \leq \text{TREN}\uparrow)$$

Más definiciones de CSPs

Soluciones

— — —

- Una **asignación** asocia un valor a algunas o todas las variables

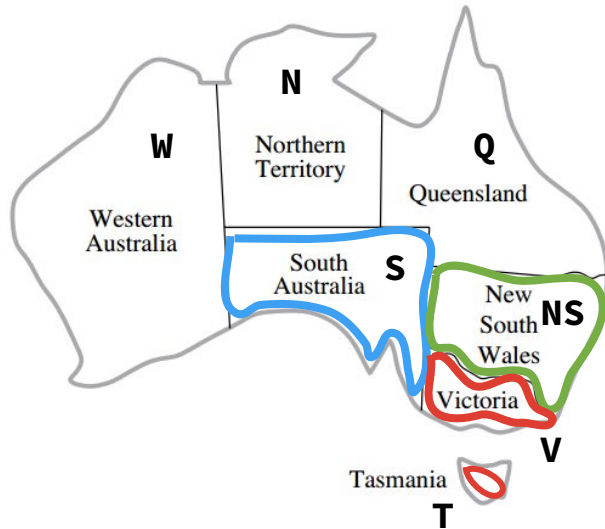
$$\{X_i = v_i, X_j = v_j, \dots\}$$

Cada variable debe tomar un valor de su dominio.

- Una asignación es **completa** si asigna un valor a todas las variables, de lo contrario es **parcial**.
- La asignación es **consistente** (o legal) si no viola ninguna restricción, de lo contrario es **inconsistente** (o ilegal).
- Una **solución** es una asignación completa y consistente.

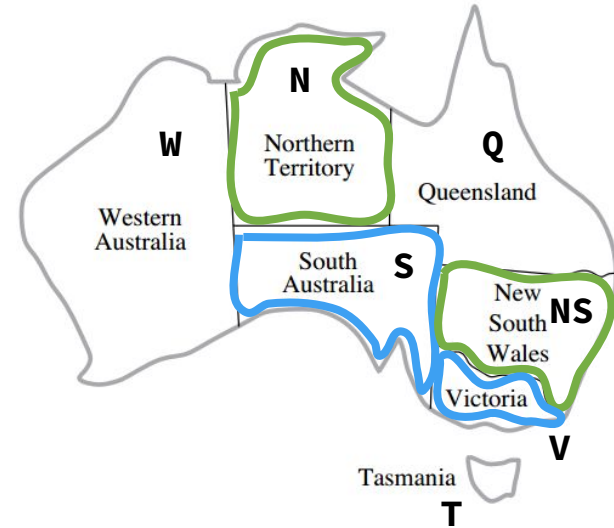


Ejemplos – Coloreo de Australia



{NS = ●, V = ●, S = ●, T = ●}

Asignación parcial y consistente.



{N = ●, NS = ●, V = ●, S = ●}

Asignación parcial e inconsistente
(viola $\langle (S,V), S \neq V \rangle$).

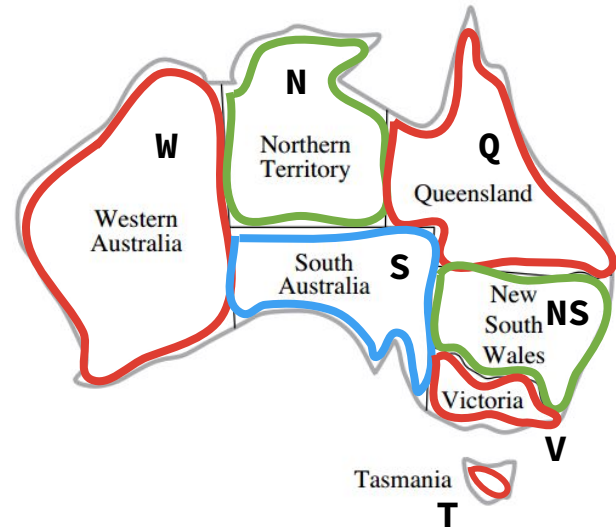


Ejemplos – Coloreo de Australia

Existen muchas soluciones, por ej:

{W=●, N=●, Q=●, NS=●,
V=●, S=●, T=●}

Asignación completa y
consistente.





Ejemplo – Scheduling

— — —

- Una asignación parcial y consistente:

$\{\text{TREN}_{\uparrow} = 0, \quad \text{RUEDA}_{\kappa} = 10, \text{RUEDA}_{\nearrow} = 10, \text{TUERCA}_{\kappa} = 11,$
 $\text{TUERCA}_{\nearrow} = 11, \quad \text{TAZA}_{\kappa} = 13, \text{TAZA}_{\nearrow} = 13, \text{INSPEC} = 14\}$

- Una asignación parcial e inconsistente:

$\{\text{TREN}_{\uparrow} = 0, \quad \text{RUEDA}_{\kappa} = 12, \text{TUERCA}_{\kappa} = 10\}$

Viola la restricción $\langle (\text{RUEDA}_{\kappa}, \text{TUERCA}_{\kappa}), \text{RUEDA}_{\kappa+1} \leq \text{TUERCA}_{\kappa} \rangle$



Ejemplo – Scheduling

— — —

- Una solución (no es única):

$\{\text{TREN}\uparrow = 0,$	$\text{RUEDA}\kappa = 10,$	$\text{RUEDA}\nearrow = 10,$	$\text{TUERCA}\kappa = 11,$
$\text{TUERCA}\nearrow = 11,$	$\text{TAZA}\kappa = 13,$	$\text{TAZA}\nearrow = 13,$	
$\text{TREN}\downarrow = 10,$	$\text{RUEDA}\searrow = 20,$	$\text{RUEDA}\swarrow = 20,$	$\text{TUERCA}\searrow = 21,$
$\text{TUERCA}\swarrow = 21,$	$\text{TAZA}\searrow = 23,$	$\text{TAZA}\swarrow = 23,$	
$\text{INSPEC} = 24\}$			

Clasificación de variables

Según su dominio, una variable se clasifica en:

- **Discreta finita**. $D_i = \{v_1, \dots, v_n\}$, para algún natural n .
- **Discreta infinita**. Por ejemplo, $D_i = \text{Naturales}$.
- **Continua**. Por ejemplo, $D_i = \text{Reales}$.

En variables con dominios infinitos no es posible en general expresar las restricciones por extensión.

Nos limitaremos a variables discretas finitas.

Clasificación de restricciones

— — —

- **Unaria**. Restringe el valor de una única variable. Por ejemplo:

$$\langle (S), S \neq \bullet \rangle$$

- **Binaria**. Relaciona dos variables. Por ejemplo:

$$\langle (S, N), S \neq N \rangle$$

- **Global**. Relaciona un número arbitrario de variables. Por ejemplo:

$$\langle (W, N, S), Alldiff(W, N, S) \rangle$$

donde $Alldiff(W, N, S)$ significa que las variables W, N, S deben tener valores mutuamente diferentes.

Clasificación de restricciones

— — —

Observación 1. Toda restricción unaria puede eliminarse de un CSP modificando adecuadamente el dominio de la variable involucrada.

 **Ejemplo - Scheduling.** La variable INSPEC tiene dominio

$$D_{\text{INSEPC}} = \{0, \dots, 30\}.$$

Entonces, la restricción unaria

$$\langle (\text{INSPEC}), \text{INSPEC} \leq 27 \rangle \text{ o bien } \langle \text{INSPEC}, \{0, \dots, 27\} \rangle$$

puede eliminarse cambiando el dominio de INSPEC a

$$D_{\text{INSEPC}} = \{0, \dots, 27\}.$$

Clasificación de restricciones

Observación 2. En general, es posible transformar una restricción global en un conjunto de restricciones binarias.

 **Ejemplo - Coloreo de Australia.** La restricción global

$$\langle (W, N, S), \text{Alldiff}(W, N, S) \rangle$$

es equivalente a las siguientes 3 restricciones binarias:

$$\langle (W, N), W \neq N \rangle, \langle (W, S), W \neq S \rangle, \langle (N, S), N \neq S \rangle$$

Clasificación de restricciones

— — —

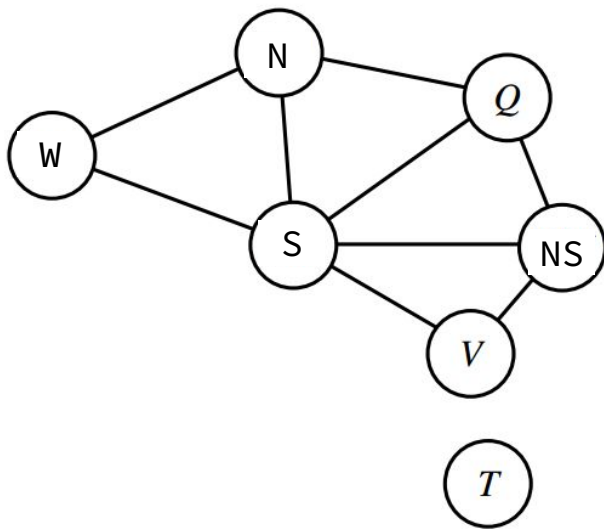
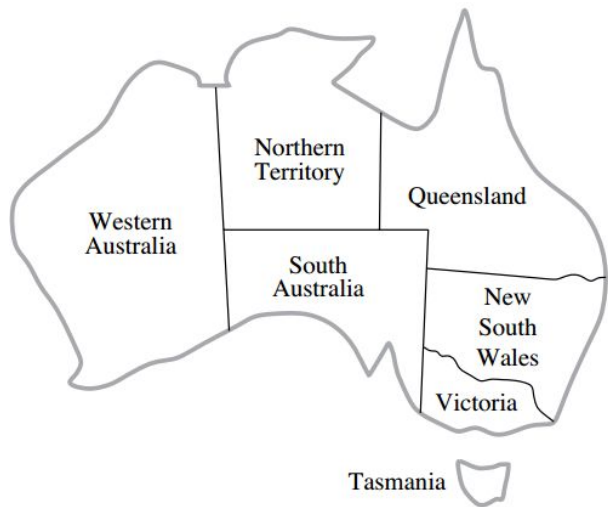
En muchas de las definiciones que estudiaremos se asume que el CSP tiene únicamente restricciones binarias.

A partir de las observaciones anteriores, se deduce:

Observación 3. Todo CSP con restricciones unarias, binarias o globales se puede transformar en un CSP equivalente que tiene únicamente restricciones binarias.

Grafo de restricciones

Los CSP suelen visualizarse con **grafos de restricciones** – donde los nodos son variables y los arcos conectan todo par de variables que participan en una restricción binaria.



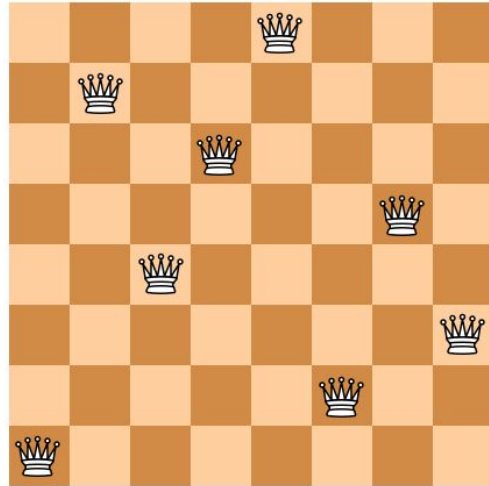
En este caso, dos vértices están conectados si y sólo si sus regiones son limítrofes.



Ejercicio

— — —

Formular como CSP el problema de las 8-reinas.





Propuesta de formulación CSP

— — —

- **Variables.** $\{F_1, \dots, F_8\}$ donde, para cada $i \in \{1, \dots, 8\}$, F_i es la fila donde se ubica la reina de la columna i .
- **Dominios.** $D_i = \{1, \dots, 8\}$ para toda variable F_i .
- **Restricciones.**
 - No hay dos reinas en una misma columna.  (gratis)
 - No hay dos reinas en una misma fila:
 $\langle (F_i, F_j), F_i \neq F_j \rangle$ para todo $1 \leq i < j \leq 8$ (28 restr. binarias)
O bien: $\langle (F_1, \dots, F_8), Alldiff(F_1, \dots, F_8) \rangle$ (1 restr. global)
 - No hay dos reinas en una misma diagonal (más difícil):
 $\langle (F_i, F_j), j - i \neq |F_i - F_j| \rangle$ para todo $1 \leq i < j \leq 8$ (28 restr. binarias).

¿Cómo resolver un CSP? 🤔

¿Es posible aplicar los algoritmos de búsqueda que ya hemos visto? ¿Cómo? 🤔

Búsqueda clásica

Formulación para CSPs

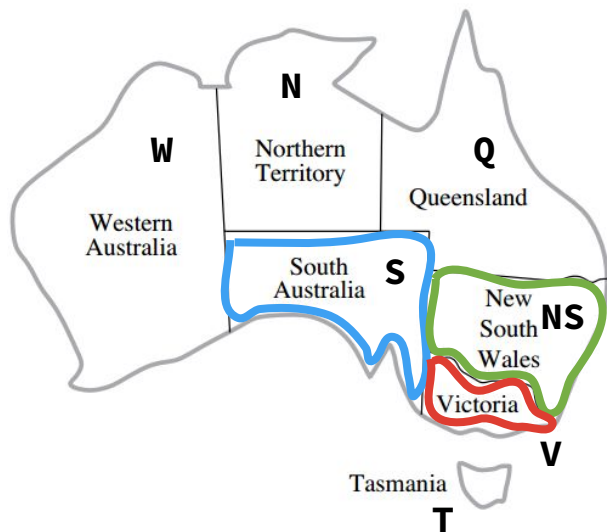
Vamos a convertir un CSP en un problema de búsqueda.

- Los **estados** son todas las asignaciones consistentes (parciales o completas).
- Las **acciones** son pares

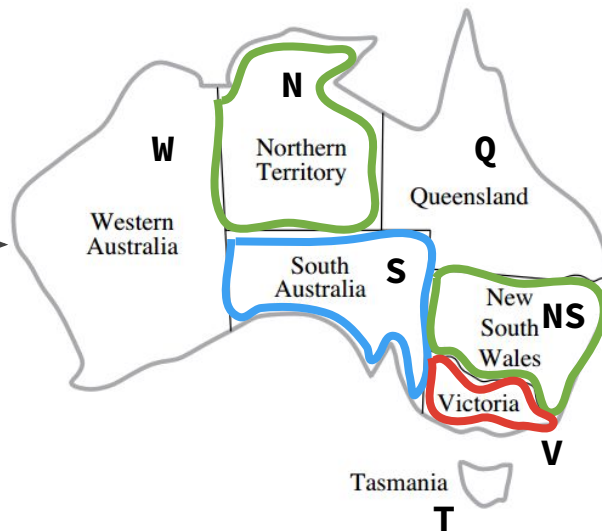
$$(X_i, v_i) \in X \times D_i$$

que representa asignar a la variable X_i el valor v_i de su dominio.

Formulación para CSPs



Pintar a N
con ●



Representación.

$$\{NS = \text{green}, V = \text{red}, S = \text{blue}\} \xrightarrow{(N, \text{green})} \{N = \text{green}, NS = \text{green}, V = \text{red}, S = \text{blue}\}$$

Formulación para CSPs

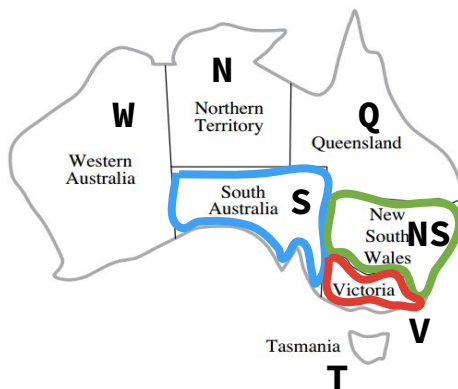
— — —

- **Estado inicial.** Asignación vacía {}.
- **Acciones.** Dada una asignación parcial $\{X_i = v_i, X_j = v_j, \dots\}$

$\text{ACCIONES}(\{X_i = v_i, X_j = v_j, \dots\}) =$

$\{(X_k, v_k) : X_k \in X, v_k \in D_k, X_k \text{ no fue asignada previamente, } X_k = v_k \text{ mantiene la asignación consistente}\}$

 **Ejemplo.**



$s = \{NS = \text{green}, V = \text{red}, S = \text{blue}\}$

$\text{ACCIONES}(s) = \{$
 $(W, \text{red}), (W, \text{green}),$
 $(N, \text{red}), (N, \text{green}),$
 $(Q, \text{red}),$
 $(T, \text{red}), (T, \text{green}), (T, \text{blue})\}$

Formulación para CSPs

— — —

- **Estado inicial.** Asignación vacía {}.
- **Acciones.** Dada una asignación parcial $\{X_i = v_i, X_j = v_j, \dots\}$
 $\text{ACCIONES}(\{X_i = v_i, X_j = v_j, \dots\}) =$
 $\{(X_k, v_k) : X_k \in X, v_k \in D_k, X_k \text{ no fue asignada previamente, } X_k = v_k \text{ mantiene la asignación consistente}\}$
- **Modelo de transiciones.** Dada una asignación $\{X_i = v_i, X_j = v_j, \dots\}$ y una acción (X_k, v_k) ,
 $\text{RESULTADO}(\{X_i = v_i, X_j = v_j, \dots\}, (X_k, v_k)) = \{X_i = v_i, X_j = v_j, \dots, X_k = v_k\}$
- **Test objetivo.** ¿La asignación es completa?
- **Costo de camino.** Es la suma de los costos individuales de las acciones que realiza. El costo individual de cada acción es 1.

Formulación para CSPs

— — —

¿Qué pasaría si aplicamos un algoritmo de búsqueda de árbol sobre esta formulación? 🤔

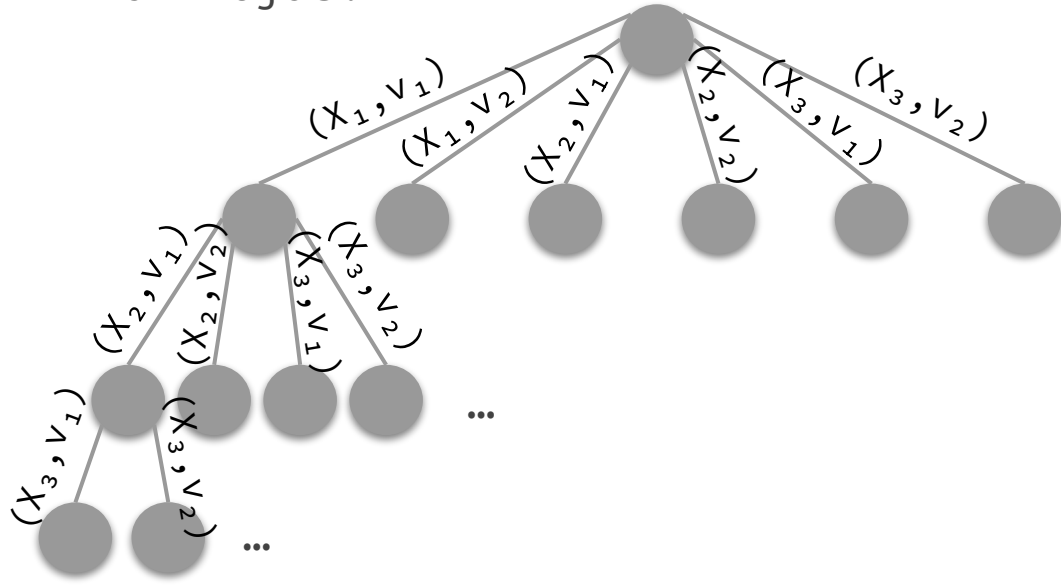
- **Estado inicial.** Asignación vacía {}.
- **Acciones.** Dada una asignación parcial $\{X_i = v_i, X_j = v_j, \dots\}$
 $\text{ACCIONES}(\{X_i = v_i, X_j = v_j, \dots\}) =$
 $\{(X_k, v_k) : X_k \in X, v_k \in D_k, X_k \text{ no fue asignada previamente, } X_k = v_k \text{ mantiene la asignación consistente}\}$
- **Modelo de transiciones.** Dada una asignación $\{X_i = v_i, X_j = v_j, \dots\}$ y una acción (X_k, v_k) ,
 $\text{RESULTADO}(\{X_i = v_i, X_j = v_j, \dots\}, (X_k, v_k)) = \{X_i = v_i, X_j = v_j, \dots, X_k = v_k\}$
- **Test objetivo.** ¿La asignación es completa?
- **Costo de camino.** Es la suma de los costos individuales de las acciones que realiza. El costo individual de cada acción es 1.



Ejemplo

Sea un CSP con variables $X = \{X_1, X_2, X_3\}$ con dominio $D_1 = D_2 = D_3 = \{v_1, v_2\}$. El número de asignaciones completas es $2^3=8$ pero el árbol de búsqueda tiene $3!.2^3 = 64$ hojas.

1. $\{X_1=v_1, X_2=v_1, X_3=v_1\}$
2. $\{X_1=v_1, X_2=v_1, X_3=v_2\}$
3. $\{X_1=v_1, X_2=v_2, X_3=v_1\}$
4. $\{X_1=v_1, X_2=v_2, X_3=v_2\}$
5. $\{X_1=v_2, X_2=v_1, X_3=v_1\}$
6. $\{X_1=v_2, X_2=v_1, X_3=v_2\}$
7. $\{X_1=v_2, X_2=v_2, X_3=v_1\}$
8. $\{X_1=v_2, X_2=v_2, X_3=v_2\}$



Tamaño del árbol de búsqueda

De manera más general, considerar un CSP con n variables, cada una con un dominio de cardinal d .

- El número de asignaciones completas está dado por el producto de los cardinales de los dominios:

$$|D_1| \cdot |D_2| \cdot \dots \cdot |D_n| = \underbrace{d \cdot d \cdot \dots \cdot d}_{n \text{ veces}} = \mathbf{d}^n$$

- En el árbol de búsqueda el factor de ramificación en la raíz es $n \cdot d$, en el siguiente nivel es $(n-1) \cdot d$ y así... Este árbol tiene $\mathbf{n! \cdot d^n}$ hojas.

Tamaño del árbol de búsqueda

Supongamos que $d = 2$ (cardinal de los dominios) y comparemos estos números al incrementar n (número de variables):

n	d^n	$n!.d^n$
6	64	3840
8	256	46080
10	1024	3715891200
12	4096	$1,96 \times 10^{12}$

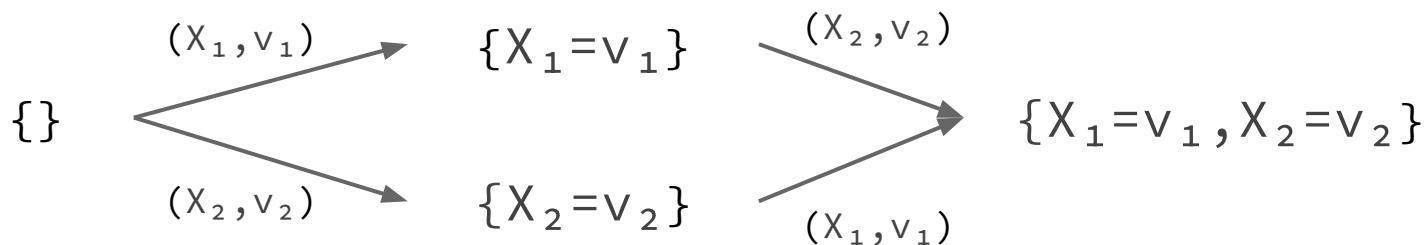
¿A qué se debe la diferencia? 🤔

y la diferencia es más significativa con el aumento de d .

Tamaño del árbol de búsqueda

La diferencia se debe a la gran cantidad de **caminos redundantes**.

📖 **Ejemplo.** La asignación $\{X_1=v_1, X_2=v_2\}$ se puede obtener como:



Conmutatividad

— — —

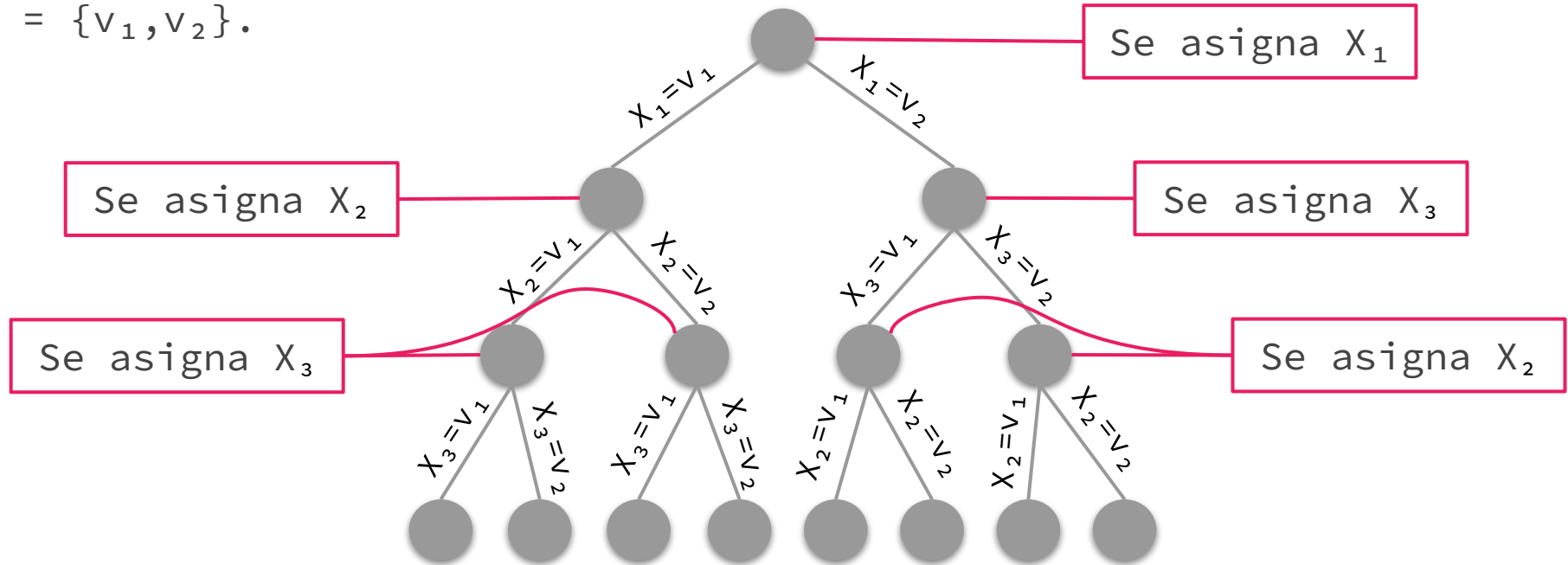
- Los CSPs tienen la propiedad de **conmutatividad**: **si se aplican las mismas acciones pero en otro orden, se obtiene la misma asignación.**
- Entonces basta con asignar una única variable en cada nodo del árbol de búsqueda.



Ejemplo

Ahora hay tantas hojas
como asignaciones
completas 👍👍

Sea un CSP con variables $X = \{X_1, X_2, X_3\}$ con dominio $D_1 = D_2 = D_3 = \{v_1, v_2\}$.



Búsqueda vuelta atrás

Backtracking search

La **búsqueda vuelta atrás** es una variación de **DFS** que elige valores para **una variable a la vez**.

En cada nodo del árbol de búsqueda se elige una variable no asignada y se genera un hijo por cada **valor consistente** para esa variable.

Retrocede cuando la variable no tiene ningún valor consistente para asignar.

— — —



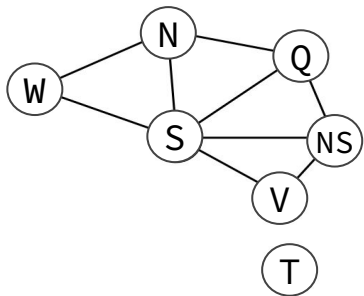
Ejemplo

-- --

Vamos a aplicar la búsqueda vuelta atrás para resolver el problema de coloreo de Australia.



Ejemplo – Coloreo de Australia



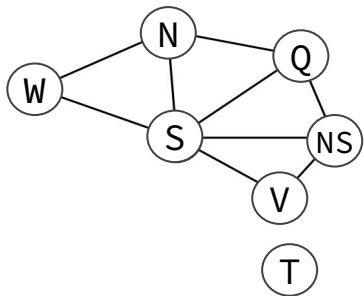
Comenzamos por el estado inicial, es decir, ninguna variable asignada.

n_0
{ }

Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	$\{n_0\}$	-
1			
2			
3			
4			
5			
6			
7			
8			
9			



Ejemplo – Coloreo de Australia



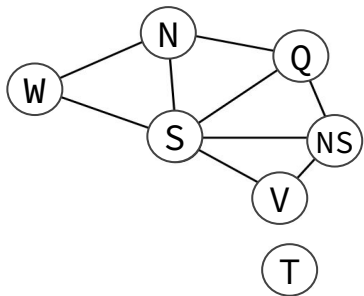
Supongamos
que se elige
asignar a W.

Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	
2			
3			
4			
5			
6			
7			
8			
9			

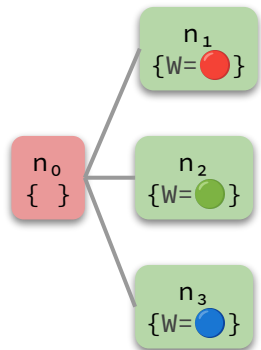
n_0
 $\{\}$



Ejemplo – Coloreo de Australia



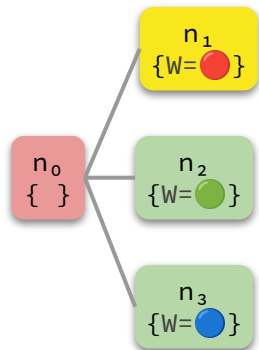
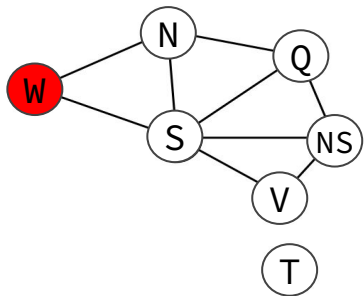
N° it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2			
3			
4			
5			
6			
7			
8			
9			



W que tiene 3 valores consistentes en su dominio: ●, ● y ●. Creamos 3 hijos, uno por cada acción (w, ●), (w, ●), (w, ●). **A diferencia de DFS, estos son los únicos hijos de n₀ generados.**



Ejemplo – Coloreo de Australia



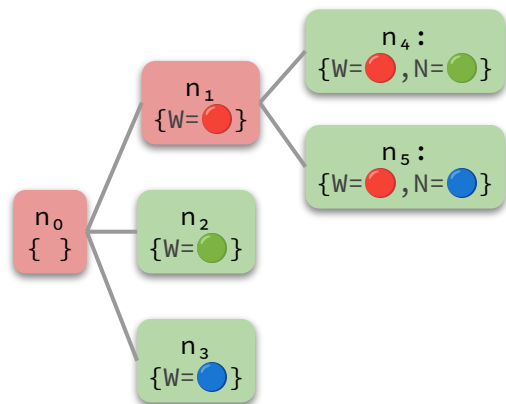
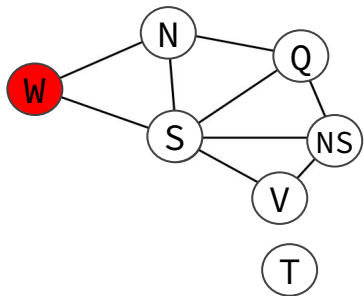
Elegimos de la frontera el nodo con la **mayor profundidad.**

Hay un empate entre n_1 , n_2 y n_3 . A priori la búsqueda vuelta atrás no tiene una regla de desempate, podemos elegir cualquiera de ellos. Elegimos por ejemplo a n_1 . Supongamos que se elige asignar a N.

N° it.	Nodo actual	Frontera antes	Frontera después
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2, n_3\}$
2	n_1	$\{n_2, n_3\}$	
3			
4			
5			
6			
7			
8			
9			



Ejemplo – Coloreo de Australia

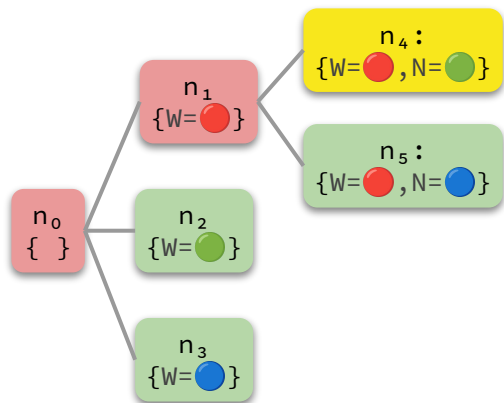
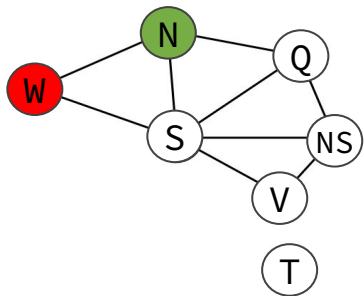


N que tiene 2 valores consistentes en su dominio ● y ● (la acción (N, ●) conduce a una asignación inconsistente pues viola la restricción $W \neq N$).

Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3			
4			
5			
6			
7			
8			
9			



Ejemplo – Coloreo de Australia

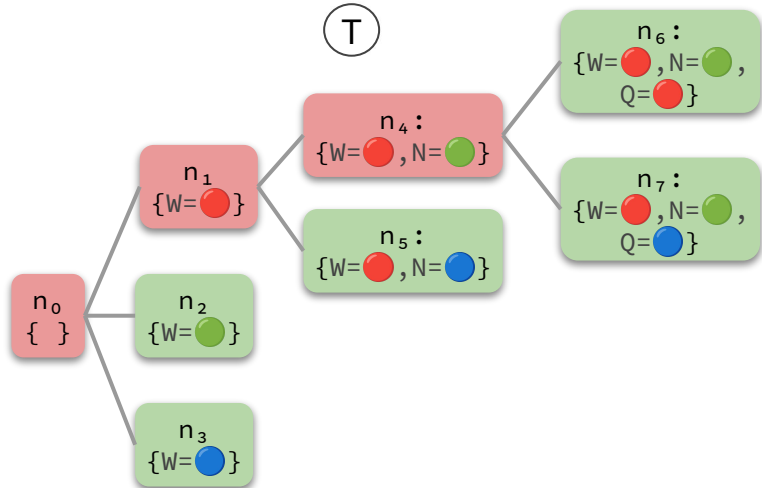
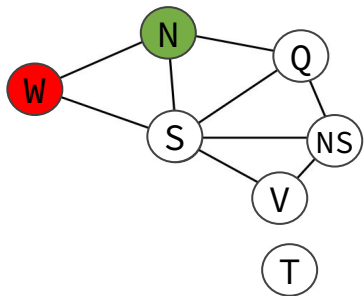


Elegimos de la frontera el nodo con la **mayor profundidad**. Hay un empate entre n_4 y n_5 . Elegimos por ejemplo a n_4 . Supongamos que se elige asignar a Q.

Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	$\{n_0\}$	-
1	n_0	$\{ \}$	$\{n_1, n_2, n_3\}$
2	n_1	$\{n_2, n_3\}$	$\{n_2, n_3, n_4, n_5\}$
3	n_4	$\{n_2, n_3, n_5\}$	
4			
5			
6			
7			
8			
9			



Ejemplo – Coloreo de Australia

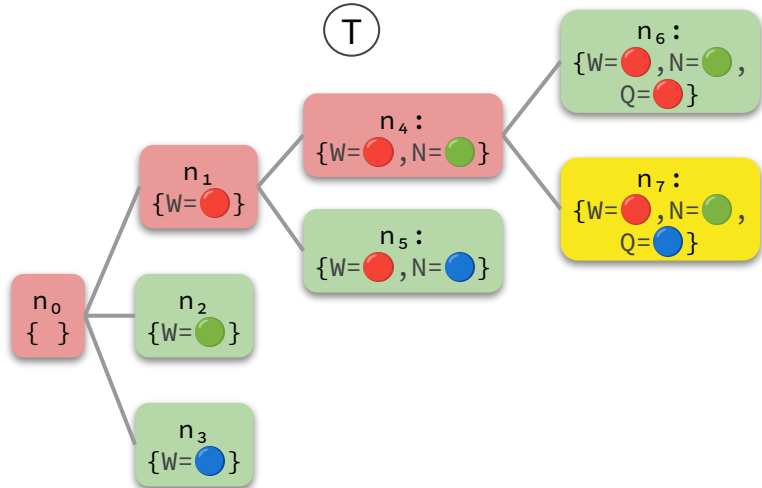
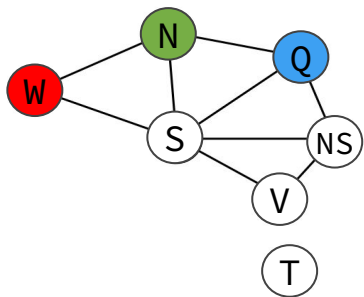


Q que tiene 2 valores consistentes en su dominio ● y ● (la acción (Q, verde) conduce a una asignación inconsistente pues viola la restricción $N \neq Q$).

Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	$\{n_0\}$	-
1	n_0	$\{ \}$	$\{n_1, n_2, n_3\}$
2	n_1	$\{n_2, n_3\}$	$\{n_2, n_3, n_4, n_5\}$
3	n_4	$\{n_2, n_3, n_5\}$	$\{n_2, n_3, n_5, n_6, n_7\}$
4			
5			
6			
7			
8			
9			



Ejemplo – Coloreo de Australia

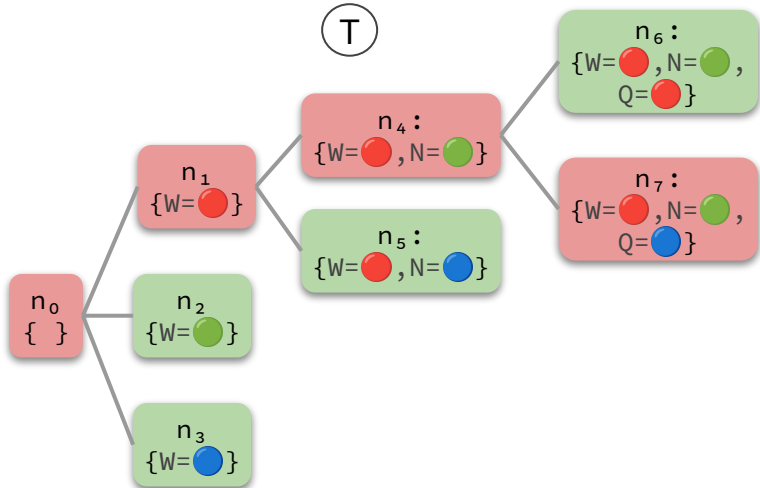
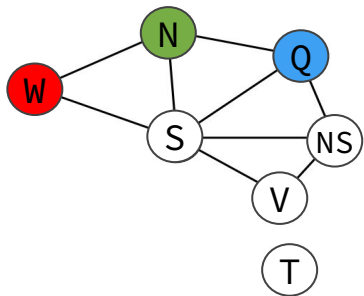


Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2, n_3\}$
2	n_1	$\{n_2, n_3\}$	$\{n_2, n_3, n_4, n_5\}$
3	n_4	$\{n_2, n_3, n_5\}$	$\{n_2, n_3, n_5, n_6, n_7\}$
4	n_7	$\{n_2, n_3, n_5, n_6\}$	
5			
6			
7			
8			
9			

Elegimos de la frontera el nodo con la **mayor profundidad**. Hay un empate entre n_6 y n_7 . Elegimos a n_7 (nos conducirá a un ejemplo un poco más interesante). Supongamos que se elige asignar a S.



Ejemplo – Coloreo de Australia

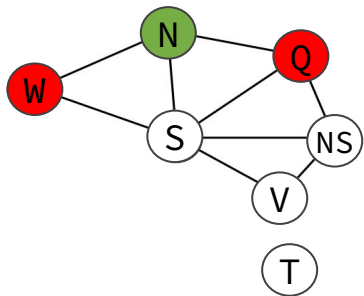


Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	$\{n_0\}$	-
1	n_0	$\{ \}$	$\{n_1, n_2, n_3\}$
2	n_1	$\{n_2, n_3\}$	$\{n_2, n_3, n_4, n_5\}$
3	n_4	$\{n_2, n_3, n_5\}$	$\{n_2, n_3, n_5, n_6, n_7\}$
4	n_7	$\{n_2, n_3, n_5, n_6\}$	$\{n_2, n_3, n_5, n_6\}$
5			
6			
7			
8			
9			

Todos los valores del dominio de S hacen a la asignación inconsistente. Luego, tenemos que **retroceder** y tomar un camino alternativo.



Ejemplo – Coloreo de Australia

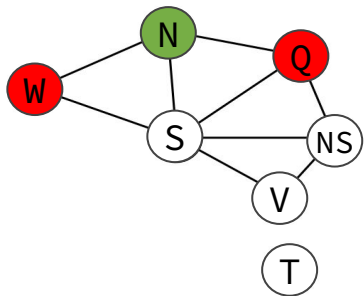


Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	
6			
7			
8			
9			

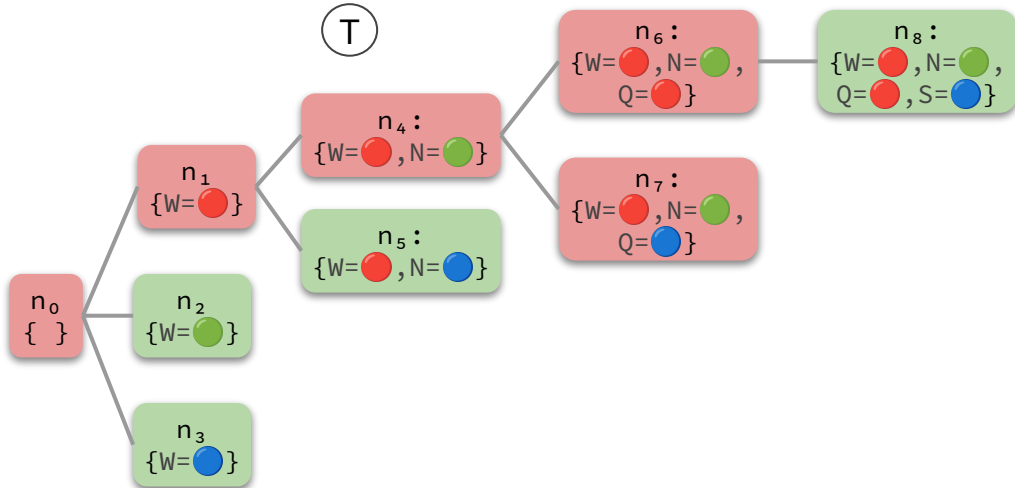
Elegimos de la frontera el nodo con la **mayor profundidad**, es decir a n₆. Supongamos que se elige asignar a S (esta decisión es independiente de la que tomamos en el nodo n₇)



Ejemplo – Coloreo de Australia



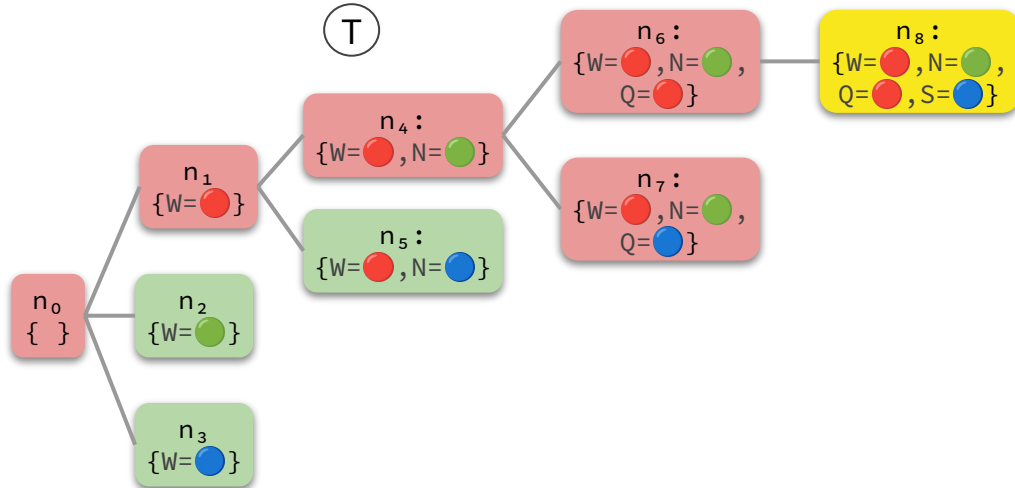
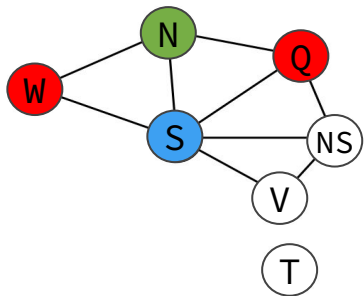
Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₈ }
6			
7			
8			
9			



S tiene a ● como único valor consistentes en su dominio (las acciones (S,●) y (S,●) conducen a asignaciones inconsistente pues violan las restricciones $N \neq S$ o $Q \neq S$).



Ejemplo – Coloreo de Australia

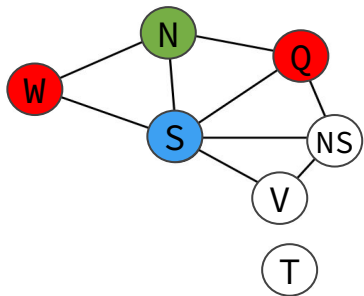


Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₈ }
6	n ₈	{n ₂ , n ₃ , n ₅ }	
7			
8			
9			

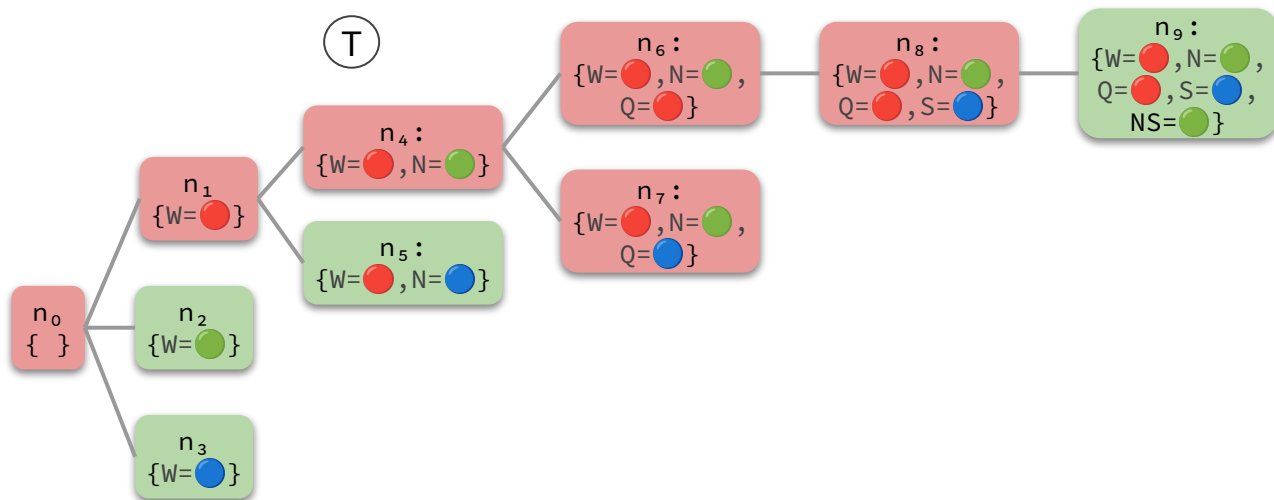
Elegimos de la frontera el nodo con la **mayor profundidad**, es decir a n₈. Supongamos que se elige asignar a NS. Continuamos así por algunas iteraciones más...



Ejemplo – Coloreo de Australia

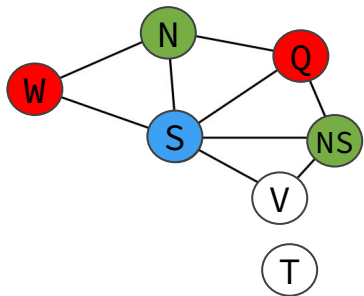


N° it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ ,n ₂ ,n ₃ }
2	n ₁	{n ₂ ,n ₃ }	{n ₂ ,n ₃ ,n ₄ ,n ₅ }
3	n ₄	{n ₂ ,n ₃ ,n ₅ }	{n ₂ ,n ₃ ,n ₅ ,n ₆ ,n ₇ }
4	n ₇	{n ₂ ,n ₃ ,n ₅ ,n ₆ }	{n ₂ ,n ₃ ,n ₅ ,n ₆ }
5	n ₆	{n ₂ ,n ₃ ,n ₅ }	{n ₂ ,n ₃ ,n ₅ ,n ₈ }
6	n ₈	{n ₂ ,n ₃ ,n ₅ }	{n ₂ ,n ₃ ,n ₅ ,n ₉ }
7			
8			
9			

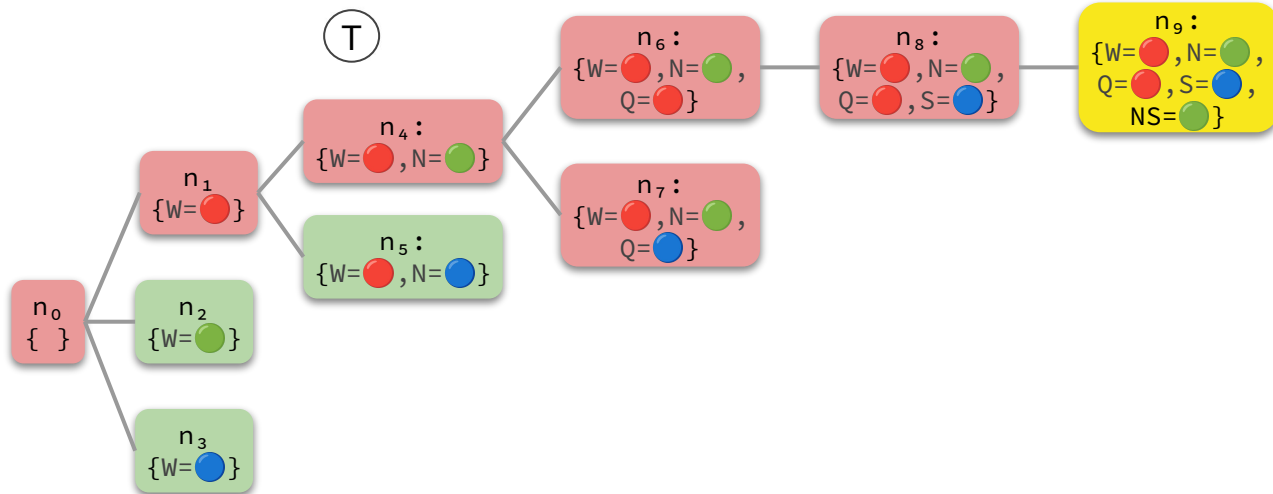




Ejemplo – Coloreo de Australia



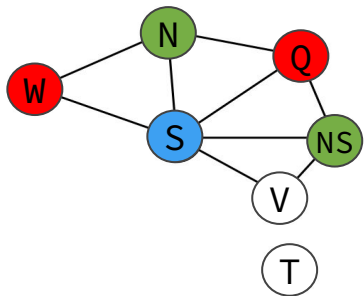
Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₈ }
6	n ₈	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₉ }
7	n ₉	{n ₂ , n ₃ , n ₅ }	
8			
9			



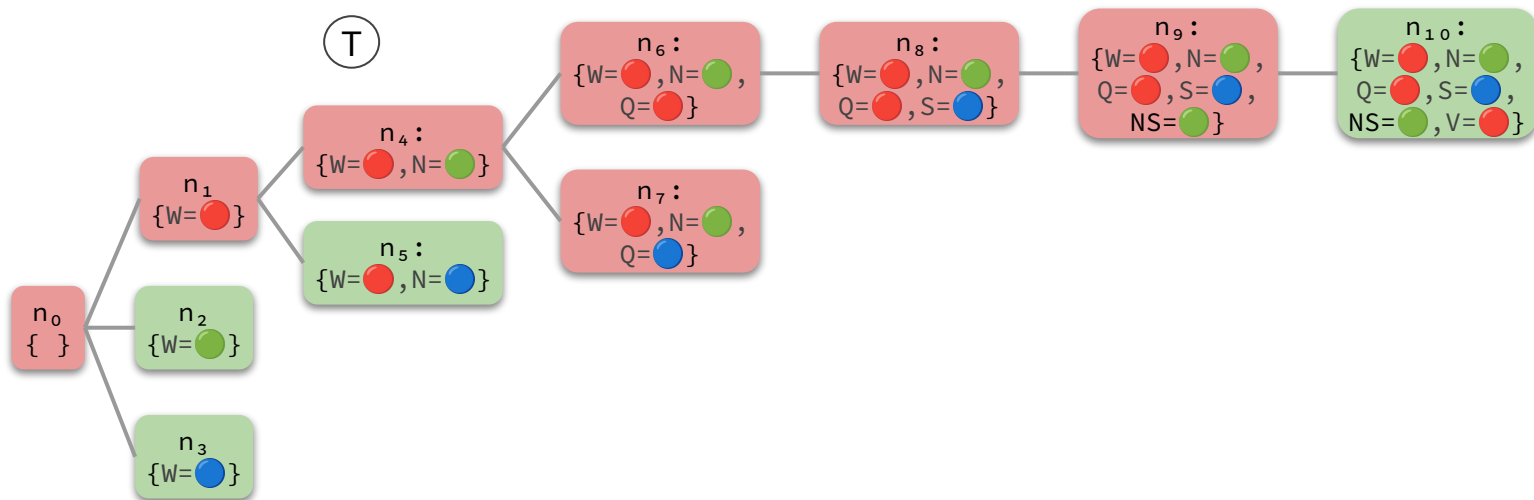
Supongamos
que se elige
asignar a V.



Ejemplo – Coloreo de Australia

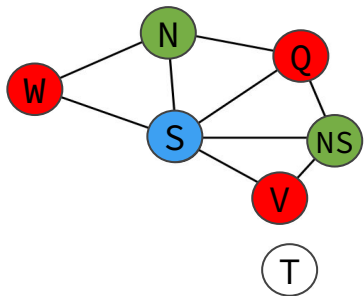


N° it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₈ }
6	n ₈	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₉ }
7	n ₉	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₁₀ }
8			
9			

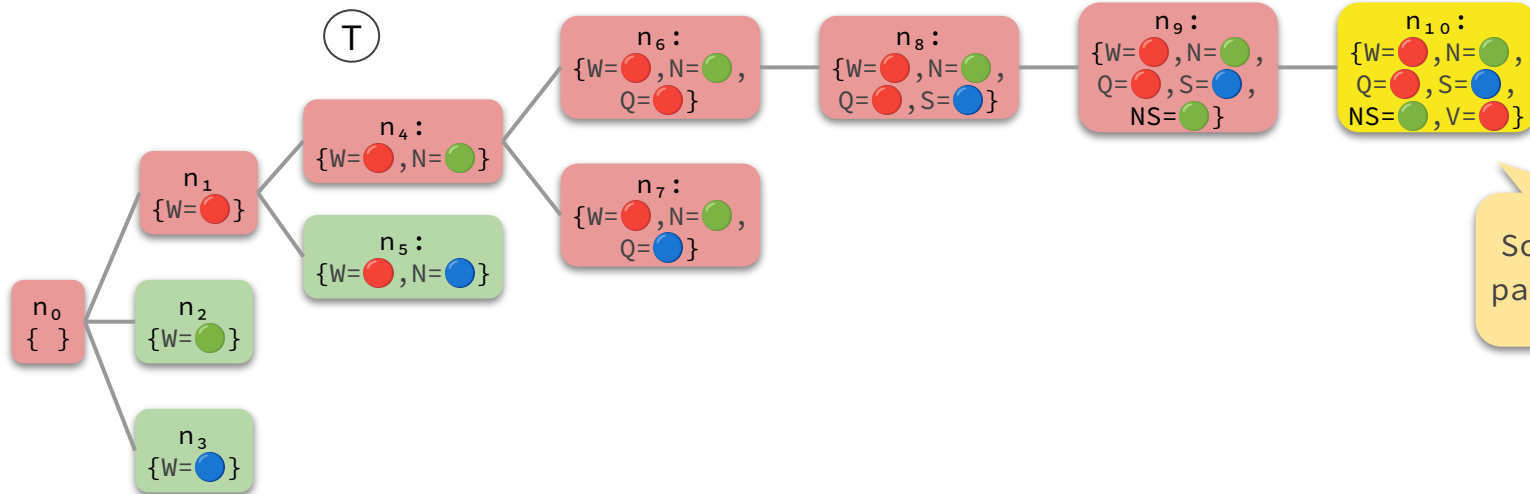




Ejemplo – Coloreo de Australia



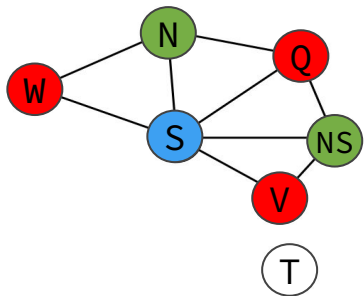
N° it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₈ }
6	n ₈	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₉ }
7	n ₉	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₁₀ }
8	n ₁₀	{n ₂ , n ₃ , n ₅ }	
9			



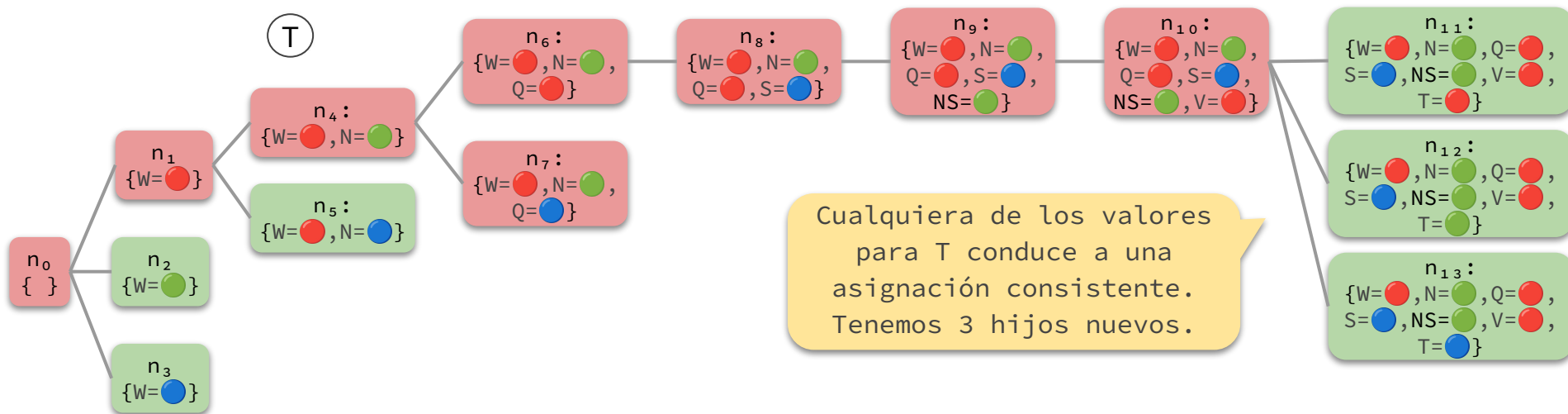
Solo queda T para asignar.



Ejemplo – Coloreo de Australia

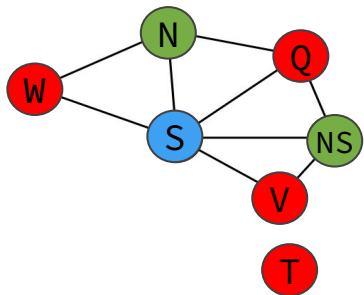


N° it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₈ }
6	n ₈	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₉ }
7	n ₉	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₁₀ }
8	n ₁₀	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₁₁ , n ₁₂ , n ₁₃ }
9			

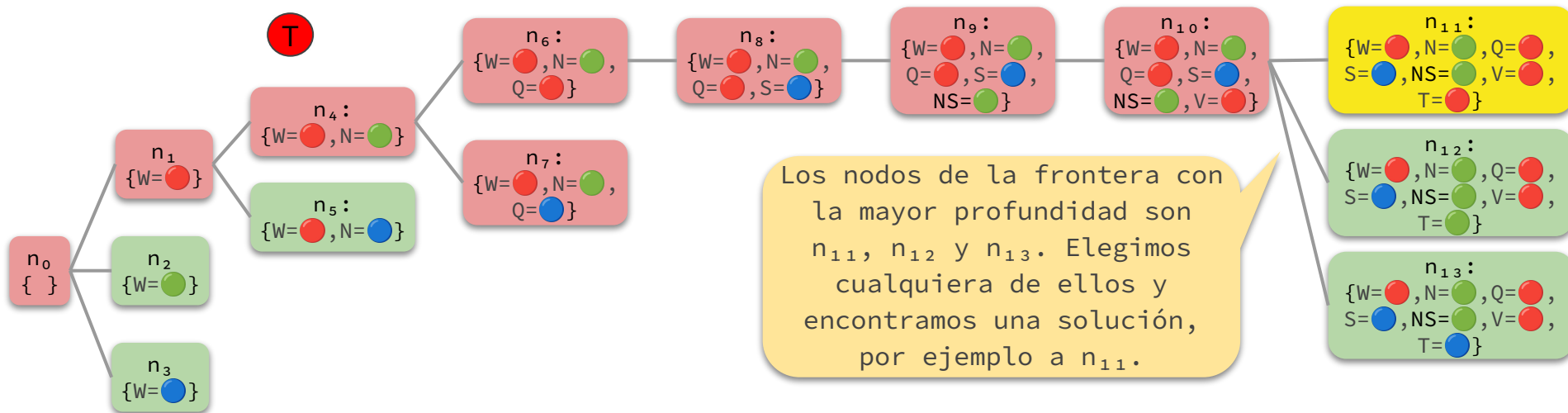




Ejemplo – Coloreo de Australia



Nº it.	Nodo actual	Frontera antes	Frontera después
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ , n ₃ }
2	n ₁	{n ₂ , n ₃ }	{n ₂ , n ₃ , n ₄ , n ₅ }
3	n ₄	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₆ , n ₇ }
4	n ₇	{n ₂ , n ₃ , n ₅ , n ₆ }	{n ₂ , n ₃ , n ₅ , n ₆ }
5	n ₆	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₈ }
6	n ₈	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₉ }
7	n ₉	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₁₀ }
8	n ₁₀	{n ₂ , n ₃ , n ₅ }	{n ₂ , n ₃ , n ₅ , n ₁₁ , n ₁₂ , n ₁₃ }
9	n ₁₁	{n ₂ , n ₃ , n ₅ , n ₁₂ , n ₁₃ }	FIN





Algoritmo de búsqueda vuelta atrás

-- -- --

- Veremos una implementación **recursiva** (no almacena explícitamente una frontera).
- Se define la función **BACKTRACK** que toma una asignación parcial, elige un valor consistente para una variable no asignada y se llama recursivamente.
- Esto se repite hasta encontrar una asignación completa.
- Si a una variable no le quedan valores consistentes para asignar, el algoritmo devuelve un fallo y la llamada recursiva termina.
- Tras el fallo, la búsqueda retrocede a la última llamada abierta y se elige un valor consistente diferente para asignar a la variable.



Algoritmo de búsqueda vuelta atrás

— — —

```
1 function BACKTRACK(csp, assign) return solución o fallo
2   if csp.es_completa(assign) then return assign
3    $X_i \leftarrow \text{ELEGIR\_VAR}(\textit{csp}, \textit{assign})$ 
4   for each v in ORDENAR_DOM(csp, assign,  $X_i$ ) do
5       assign[ $X_i$ ]  $\leftarrow v$ 
6       if csp.es_consistente(assign) then
7           res  $\leftarrow$  BACKTRACK(csp, assign)
8           if res  $\neq$  fallo then return res
9       assign.remove( $X_i$ )
10  return fallo

1 function BACKTRACKING-SEARCH(csp) return solución o fallo
2  return BACKTRACK(csp, {})
```



Algoritmo de búsqueda vuelta atrás

```
1 function BACKTRACK(csp, asign) return solución o fallo
2   if csp.es_completa(asign) then return asign
3    $X_i \leftarrow \text{ELEGIR\_VAR}(\textit{csp}, \textit{asign})$ 
4   for each  $v$  in ORDENAR_DOM(csp, asign,  $X_i$ ) do
5     asign[ $X_i$ ]  $\leftarrow v$ 
6     if csp.es_consistente(asign) then
7       res  $\leftarrow$  BACKTRACK(csp, asign)
8       if res  $\neq$  fallo then return res
9     asign.remove( $X_i$ )
10  return fallo
```

Observación 1. ELEGIR_VAR elige una variable no asignada con algún criterio.

Observación 2. ORDENAR_DOM ordena los valores del dominio de X_i con algún criterio.

```
1 function BACKTRACKING-SEARCH(csp) return solución o fallo
2   return BACKTRACK(csp, {})
```



Algoritmo de búsqueda vuelta atrás

```
1 function BACKTRACK(csp, asign) return solución o fallo
2   if csp.es_completa(asign) then return asign
3    $X_i \leftarrow \text{ELEGIR\_VAR}(\textit{csp}, \textit{asign})$ 
4   for each v in ORDENAR_DOM(csp, asign,  $X_i$ ) do
5     asign[ $X_i$ ]  $\leftarrow v$ 
6     if csp.es_consistente(asign) then
7       res  $\leftarrow$  BACKTRACK(csp, asign)
8       if res  $\neq$  fallo then return res
9     asign.remove( $X_i$ )
10  return fallo
```

Observación 3. Si la llamada recursiva devuelve una solución, la llamante devuelve también esa solución. Si devuelve un fallo, la llamante elimina la variable recién asignada y se intenta un valor consistente diferente.

Observación 4. Tras intentar con todos los valores de su dominio y no encontrar solución, se devuelve un fallo.

```
1 function BACKTRACKING-SEARCH(csp) return solución o fallo
2   return BACKTRACK(csp, {})
```

Observación 5. La búsqueda hacia atrás simplemente llama a **BACKTRACK** sobre la asignación vacía.



Algoritmo de búsqueda vuelta atrás

-- --

- Para todo CSP, `BACKTRACKING-SEARCH` usa implícitamente el mismo estado inicial, función de acciones, modelo de transiciones y test objetivo.
- **No necesita funciones específicas del problema a resolver.**
- Garantiza **completitud** si los dominios son finitos.
- Más adelante veremos propuestas para `ELEGIR_VAR` y `ORDENAR_DOM`.

Búsqueda local

Búsqueda local

Otra alternativa para resolver un CSP general es mediante **búsqueda local**. Necesitamos:

1. **Formulación de estados completa.**
2. **Función objetivo** que alcance su máximo valor en las soluciones del CSP.

¿Cómo lo hacemos? 🤔

1. Formulación de estados completa

Los **estados** son las **asignaciones completas** (consistentes o inconsistentes), es decir, todas las variables ya tienen asignado un valor de su dominio.

El número total de estados es:

$$\prod_i |D_i|,$$

es decir, el producto de los cardinales de los dominios (siempre que sean finitos).

1. Formulación de estados completa

Las **acciones** son los pares

$$(X_i, v_i) \in X \times D_i$$

que representan **reasignar** a la variable X_i con el valor v_i de su dominio.

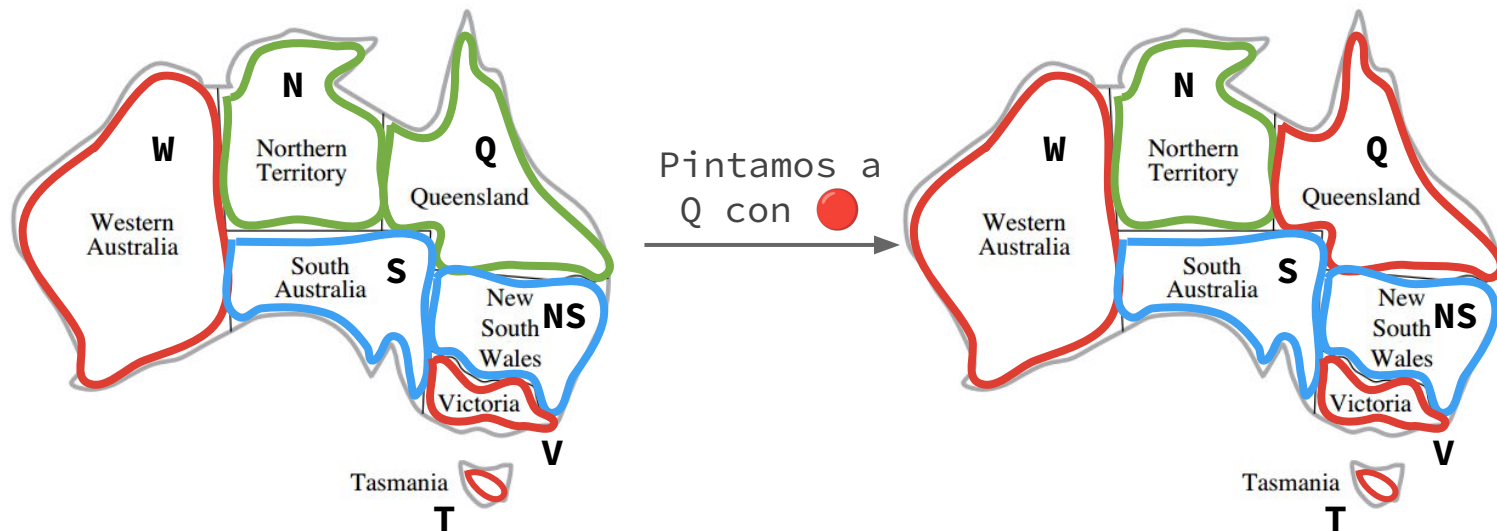
El número de sucesores en cada estado es:

$$\sum_i |D_i| - 1,$$

es decir, es posible asignar a cualquier variable con cualquier valor de su dominio diferente de su valor actual.



Ejemplos – Coloreo de Australia



Representación:

$$\{W=\text{red}, N=\text{green}, Q=\text{green}, NS=\text{blue}, V=\text{red}, S=\text{blue}, T=\text{red}\} \xrightarrow{(Q, \text{red})} \{W=\text{red}, N=\text{green}, Q=\text{red}, NS=\text{blue}, V=\text{red}, S=\text{blue}, T=\text{red}\}$$

1. Formulación de estados completa

— — —

- **Estado inicial.** Cualquier asignación completa.
- **Acciones.** Dado un estado $\{X_1=v_1, \dots, X_n=v_n\}$,
$$\text{ACCIONES}(\{X_1=v_1, \dots, X_n=v_n\}) = \{(X_i, u_i) : X_i \in X, u_i \in D_i, u_i \neq v_i\}$$
- **Modelo de transiciones.** Dado un estado $\{X_1=v_1, \dots, X_n=v_n\}$ y una acción (X_i, u_i) ,
$$\text{RESULTADO}(\{X_1=v_1, \dots, X_n=v_n\}, (X_i, u_i)) = \{X_1=v_1, \dots, \mathbf{X_i=u_i}, \dots, X_n=v_n\}$$
- **Test objetivo.** Dado un estado $\{X_1=v_1, \dots, X_n=v_n\}$, ¿es consistente?

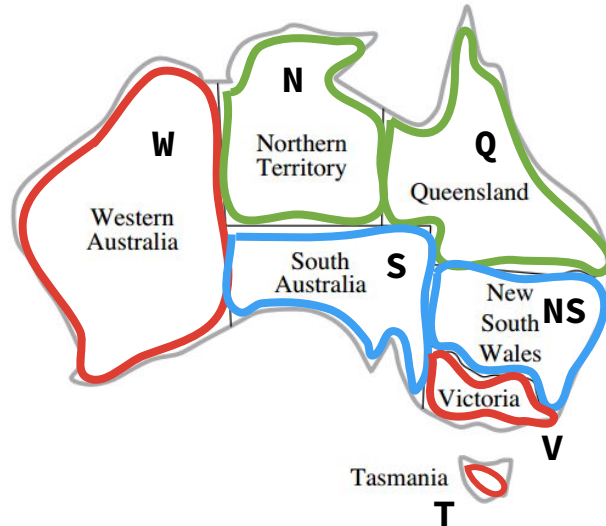
2. Función objetivo

- El objetivo es encontrar una **asignación consistente**.
- Es decir, una asignación donde el número de restricciones violadas sea 0.
- Definimos la función f tal que, para toda asignación s , $f(s)$ es el **número de restricciones violadas**.
- El objetivo es minimizar f o equivalentemente, maximizar $-f$.
- Notar que 0 es el mínimo global y $f(s) = 0$ si y sólo si s es una solución del CSP.



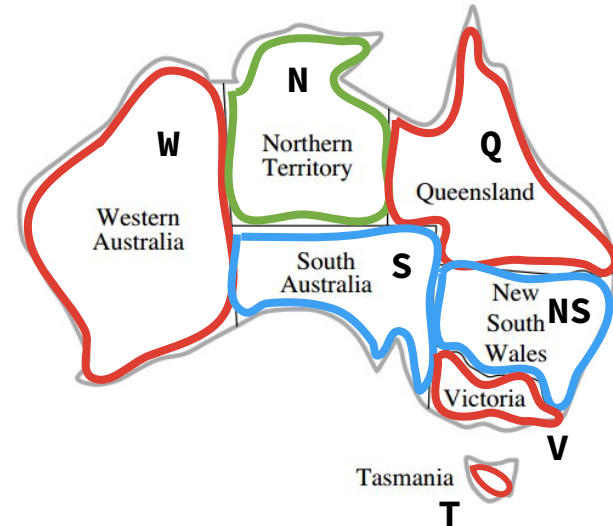
Ejemplos – Coloreo de Australia

Siempre considerando al CSP con restricciones binarias (reescribiendo las restricciones globales si las hubiera).



$$s = \{W=\text{red}, N=\text{green}, Q=\text{green}, NS=\text{blue}, V=\text{red}, S=\text{blue}, T=\text{red}\}$$

$$f(s) = 2 \text{ (se violan } N \neq Q \text{ y } S \neq NS)$$



$$s' = \{W=\text{red}, N=\text{green}, Q=\text{red}, NS=\text{blue}, V=\text{red}, S=\text{blue}, T=\text{red}\}$$

$$f(s') = 1 \text{ (se viola } S \neq NS)$$

Heurística de mínimos-conflictos

Min-conflicts heuristic

La **heurística de mínimos-conflictos** es un algoritmo de **búsqueda local** para resolver CSPs.

Usa la **formulación de estados completa** y la **función objetivo** presentadas anteriormente.

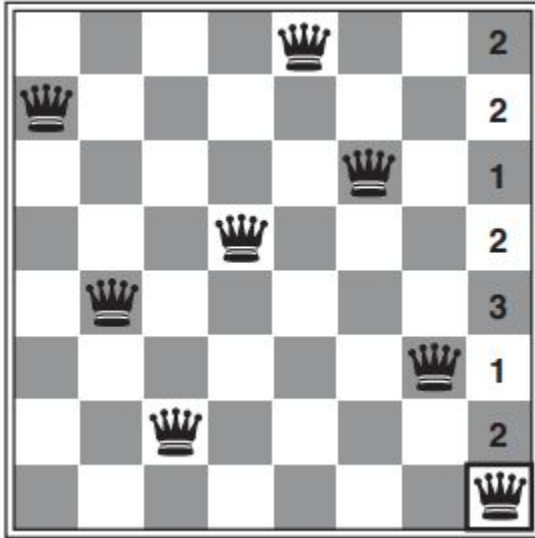
En en cada iteración, se elige al **azar** una variable conflictiva (participante en alguna restricción violada) y se le asigna el valor que minimiza el número de restricciones violadas.

— — —

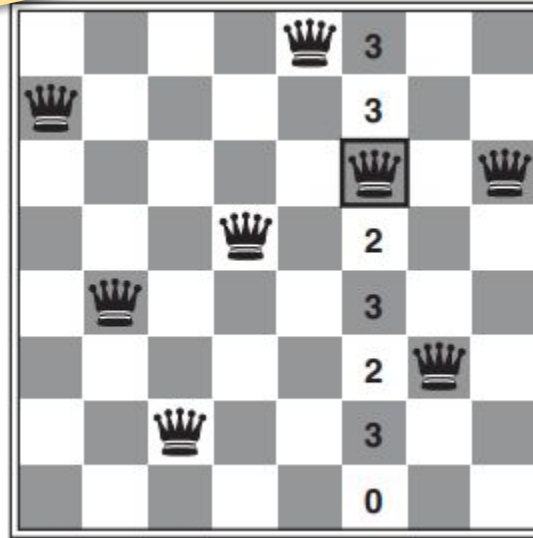


Ejemplo – 8 Reinas

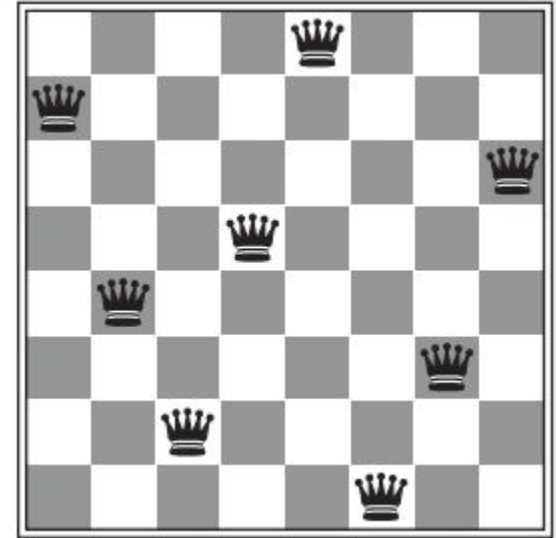
Los números de las casillas indican el valor objetivo que resulta de mover a la reina de esa columna a esa fila.



1. Estado inicial. Elegimos una reina en conflicto al azar, por ej. la de la columna 8. La movemos a su mejor posición, por ej. a la fila 3.



Elegimos una reina en conflicto al azar, por ej. la de la columna 6. La movemos a su mejor posición, es decir a la fila 8.



3. Llegamos a un estado sin reinas atacadas.
¡Encontramos una solución!



Algoritmo de mínimos-conflictos

```
1 function MIN-CONFLICTS(csp) return solución o fallo
2   actual ← csp.estado-inicial
3   while no se cumple criterio de parada do
4     if csp.es_consistente(actual) then return actual
5      $X_i \leftarrow$  elegir al azar una variable conflictiva de csp.X
6      $S_i \leftarrow \{csp.resultado(actual, (X_i, u_i)) : u_i \in D_i, u_i \neq actual[X_i]\}$ 
7     sucesor ← elegir elemento de  $S_i$  que minimiza csp.f
8     actual ← sucesor
9   return fallo
```

S_i es el subconjunto de sucesores obtenidos de aplicar cada acción (X_i, u_i) , para todo valor u_i del dominio de X_i distinto del valor actual de X_i .

Performance de MIN-CONFLICTS

- Es sorprendentemente eficaz para muchos CSPs.
 - Puede resolver el problema del millón de reinas en un promedio de 50 pasos (suponiendo una buena asignación inicial).
- El paisaje de espacio de estados de un CSP tiene usualmente muchas mesetas.
- Las técnicas vistas anteriormente para búsqueda local pueden ayudar a escapar de estas mesetas: movimientos laterales y lista tabú.



Resumen

— — —

- ❑ Un CSP representa un estado como una **asignación** de valores a un conjunto de **variables**, donde cada variable tiene un **dominio** y las condiciones que tienen que cumplir las soluciones se representan con un conjunto de **restricciones**.
- ❑ Muchos problemas importantes del mundo real se pueden escribir como CSPs.
- ❑ Es posible formular a un CSP como un **problema de búsqueda** y aplicar los algoritmos de búsqueda ya conocidos.
 - ❑ La **búsqueda hacia atrás** es un DFS que incorpora una noción de orden entre las variables y se usa habitualmente para resolver CSPs.
 - ❑ La **heurística de mínimos-conflictos** es una búsqueda local que ha tenido gran éxito para resolver CSPs.
- ❑ Existen solvers en python para CSPs.



Próximamente

— — —

Recordar que los algoritmos de búsqueda no-informada se podían mejorar incorporando funciones heurísticas derivadas de nuestro conocimiento específico del problema.

La búsqueda hacia atrás puede mejorarse significativamente sin tener **ningún tipo de conocimiento específico del problema.**

¿Cómo lo hacemos?

